GeoWild: Wildlife Territorial Conflict & Habitat Monitoring System

Samridhi Mukhopadhay	1134
Ritam Biswas	1128
Sanu Mukherjee	1135

1 Abstract

GeoWild is a web application that simulates a herd-based wildlife habitat and uses it to demonstrate two classic algorithms from computational geometry. Fifty simulated animals (elephants, zebras, giraffes and lions) move on a 100 × 100 plane. At any moment the animals' current coordinates can be given to two algorithms that we implemented by hand in Python: the **closest pair of points**, solved by divide and conquer, and **rectangular range search**, solved with a two-dimensional KD-tree.

The first algorithm answers "which two animals are closest to each other?", which in a conflict-monitoring story marks the pair most likely to be in dispute over territory. The second answers "which animals are inside this monitoring zone?". A React interface draws the habitat, shows the results, and replays each algorithm step by step from traces produced by the algorithm code itself.

All animals and movements are **simulated** from a fixed random seed; the project uses no real tracking data. For the sample state analysed in this report (tick 101) the closest pair was a zebra and a lion 0.69 units apart, found with 57 distance computations instead of the 1,225 needed to compare every pair. A monitoring zone of x 35–70, y 30–65 contained 9 animals, and the KD-tree visited 19 of its 50 nodes to find them where a linear scan would test all 50 animals.

2 Problem statement and objectives

Imagine a reserve in which rangers want to watch a group of animals. Two questions come up repeatedly, and both are questions about points on a plane:

1. **Closest pair.** Given the positions of all animals, which two are nearest to each other? The pair that is closest together is the one most likely to be in territorial conflict, so it is worth flagging.
2. **Rectangular range search.** Given a rectangular monitoring or danger zone, which animals are inside it? Rangers may want to count or list them, and to repeat the question as the rectangle is moved.

Both can be answered by a simple method. Comparing every pair of animals takes $O(n^2)$ time (1,225 pairs for 50 animals), and testing every animal against the rectangle takes $O(n)$ time for each query. These simple methods are fine for 50 animals but become slow for large data sets, which is why the two problems are standard examples in an algorithms course. They are suitable for this project because each one has a clear picture behind it: the closest-pair algorithm divides the plane and checks only a narrow strip, and the KD-tree divides the plane into regions that can be skipped. Both can therefore be drawn and followed visually.

The objectives of the project were:

- to implement closest pair (divide and conquer, O(n log n)) and a 2-D KD-tree with rectangular range search (build O(n log n), query O(√n + k)) by hand in Python, without library routines that solve these problems directly;
- to keep the algorithms independent of the simulation, so that they receive only an id and an (x, y) position for each animal;
- to verify both algorithms against brute force and a linear scan, and to report operation counts next to the theoretical complexity;
- to present the algorithms in a web interface that shows what they do on live, changing data, with step-by-step walkthroughs.

GeoWild is an educational simulation and not a real wildlife-tracking system. The animal emoji are only visual labels for the data points.

3 System architecture and implementation

GeoWild has four parts (Figure 1). A React front end shows the habitat and the algorithm walkthroughs. A FastAPI backend, written in Python, serves the data. Inside the backend a seeded simulation produces the animals' positions, and a separate algorithm layer analyses those positions on request.

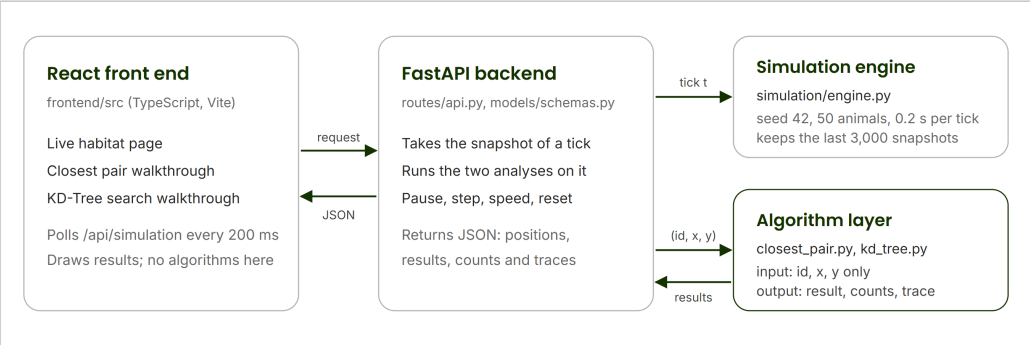


Figure 1. System architecture. The simulation produces positions; the algorithm layer receives only an id and an (x, y) position for each animal and returns results, operation counts and traces.

3.1 Technology

The front end is written in React 19 and TypeScript, built with Vite and styled with Tailwind CSS. The habitat and the algorithm views are drawn as SVG. The front end does not implement either algorithm: it only requests results and draws them. The backend uses FastAPI with Pydantic models for the responses, and the two algorithms are plain Python modules (`algorithms/closest_pair.py` and `algorithms/kd_tree.py`). There is no database; the simulation state is kept in memory.

3.2 How animals become algorithm input

The habitat is a plane with x and y running from 0 to 100, with the origin at the lower left and y increasing upwards. Each animal has an id whose first letter gives its species (for example E01 for an elephant or L04 for a lion), a position (x, y) rounded to three decimals, and a velocity used by the simulation. At every tick the simulation records a **snapshot**: the list of all 50 animals at that tick.

When an analysis is requested for a tick, the backend takes the snapshot of that tick and passes it to the algorithm. The algorithms read only `id`, `x` and `y`; species, herd membership and velocity belong to the simulation and are never used in the geometry. Because the same snapshot can be requested again by tick number, any result can be reproduced and compared with a simple method.

3.3 Backend interface

The backend advances the simulation in the background (5 ticks per second at normal speed) and exposes the endpoints in Table 1. Results for recently used ticks are kept in a small cache so that repeated requests for the same tick are cheap. A tick that has left the stored history returns a 404 error, and the interface then asks the user to take a new snapshot.

Table 1. Main backend endpoints.

Endpoint	Purpose
GET /api/simulation	Current tick, all animals with positions and velocities, the live closest pair, and the result for the monitoring zone
POST /api/simulation/control	Pause, resume, step, change speed, or restart the simulation
GET /api/closest-pair?tick=	Closest pair for a tick, with operation counts and the trace used by the walkthrough
POST /api/range-search	Rectangle bounds and tick; returns the matching animals and the visited and pruned nodes
GET /api/kdtree?tick=	The KD-tree nodes for a tick (split axis, depth, region), used to draw the partitions

3.4 Front-end data flow

The Live habitat page asks the backend for the current state about five times per second and animates the animals between updates, so the motion looks smooth. The Closest pair and KD-Tree search pages work on a single frozen snapshot: when the page opens it records the current tick and requests the analysis for exactly that tick. The picture therefore stays still while it is being studied, and the **New snapshot** button analyses the simulation as it is at that moment.

4 The simulation and the Live habitat

4.1 Movement model

The simulation exists to give the algorithms realistic, changing input. It is deterministic: it uses a single random number generator with seed 42, consumed in a fixed order, so the state at tick t depends only on the seed and t. The same seed always produces the same herds and paths. One tick is 0.2 simulated seconds.

The population is made of four species (Table 2). Elephants, zebras and giraffes move in herds; lions move alone.

Table 2. The simulated population (50 animals in 17 groups).

Species	Animals	Group size	Cruising speed (units/s)	Behaviour
Elephant	10	5	0.9	Slow herds that stay close together
Zebra	18	6	2.0	Fast herds with strong cohesion
Giraffe	12	6	1.2	Loose herds at a medium pace
Lion	10	1 (solitary)	1.6	Independent roaming paths

Movement is built from a few simple rules, chosen so that animals never jump:

- **Group heading.** Each group has a heading that changes gradually through a smoothed random turn rate, and a speed that varies slowly between about half and full of its cruising speed, like a walk and graze rhythm.
- **Edges.** When a group's centre comes within 14 units of the edge, its heading is turned back towards the middle, more strongly the closer it is.
- **Herding.** Herd members drift individually but are pulled towards the herd centre when they are more than 4 units away from it.
- **Personal space.** Two animals closer than 1.8 units push each other apart, so animals do not sit on top of each other.
- **Smooth velocity.** Each animal's velocity eases towards its target velocity rather than changing instantly.

Positions are recorded at every tick and the last 3,000 ticks (ten minutes at normal speed) are kept. The speed control (0.5× to 4×) changes only how fast ticks arrive, never what they contain, so the speed does not affect the results.

4.2 The Live habitat page

Figure 2 shows the Live habitat page. The large panel is the habitat itself: a plane with labelled x and y axes and one emoji per animal. The closest pair is marked by a red ring and a dashed red line with its distance. The translucent red rectangle is the **monitoring zone**; it can be dragged and resized, and the animals currently inside it are ringed in orange. The two panels on the right report the closest pair and the zone result, including the operation counts, and a control bar below the habitat pauses, resumes, steps and restarts the simulation and sets its speed. Both results are recomputed on the current positions, so they change as the animals move. To keep the red highlight from flickering when two pairs are almost equally close, the page changes the highlighted pair only after a new pair has stayed the closest for a short time, unless the old pair is clearly no longer the closest.

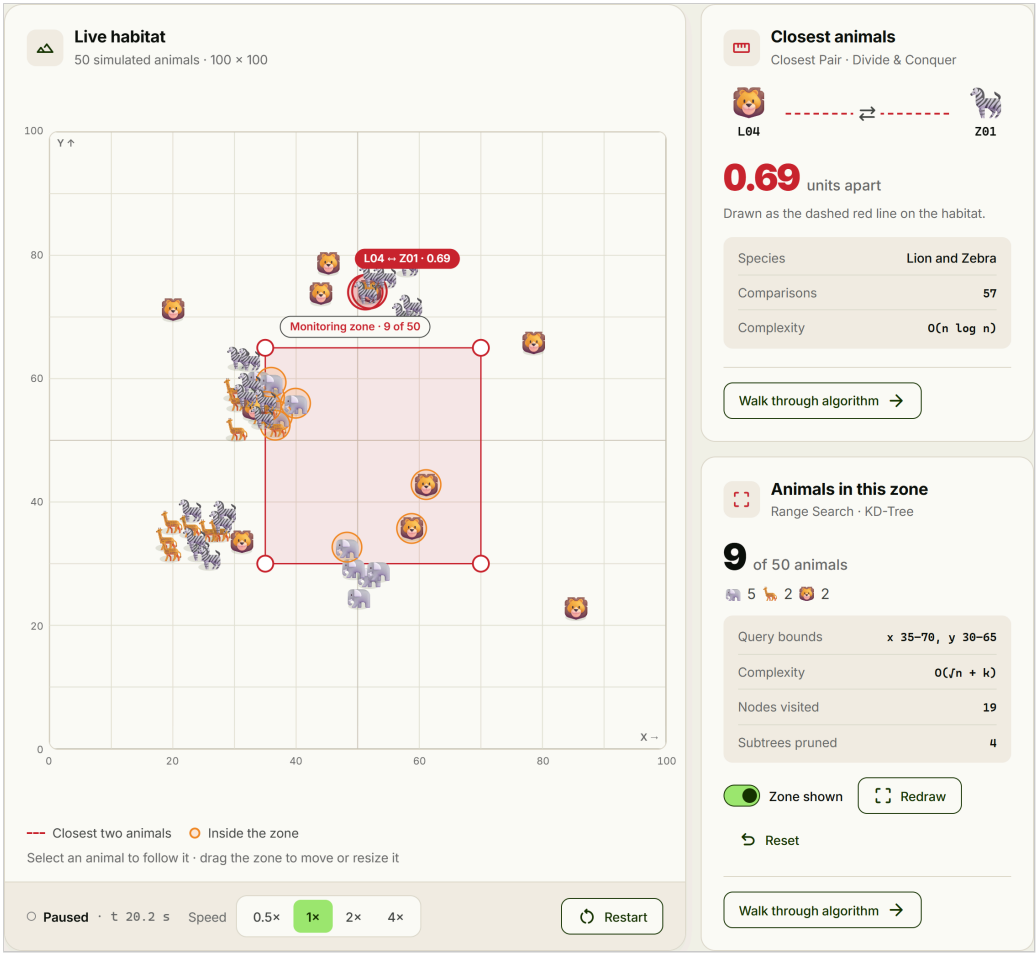


Figure 2. The Live habitat page, paused at tick 101 (t = 20.2 s). Left: the habitat with the closest pair (red) and the monitoring zone with the animals inside it ringed in orange. Right: the closest-pair and zone panels. Below: the simulation controls.

5 Closest pair of points

5.1 The problem and the simple solution

Given n points, the problem is to find the two points with the smallest distance between them. The distance between points (x_1, y_1) and (x_2, y_2) is the Euclidean distance:

$$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The simple solution computes the distance for every pair and keeps the smallest. There are $n(n-1)/2$ pairs, so the time is $O(n^2)$: 1,225 comparisons for our 50 animals, but about 5 million for 3,200 points. Divide and conquer avoids most of these comparisons. Since only the order of distances matters, the implementation compares squared distances and takes a square root only for the reported answer.

5.2 The divide-and-conquer algorithm

1. **Sort once.** The animals are sorted by x and, separately, by y. These two orders are made only once at the start and reused in every recursive call.

2. **Divide.** The animals sorted by x are split in the middle: the left half holds the smaller x-values and the right half the larger ones. The x-coordinate of the median animal is the dividing line.
3. **Solve both halves.** The closest pair of each half is found recursively. If a group has three animals or fewer (the base case), its pairs are compared directly, which is at most three comparisons.
4. **Choose the best distance.** Let d be the smaller of the two distances found in the halves. The closest pair of the whole set is either one of the pairs already found, or a pair with one animal in each half. Only the second case is still open.
5. **Build the strip.** If a pair across the dividing line is closer than d, both of its animals must be within distance d of the line. The strip contains exactly the animals whose x-distance to the line is less than d (a band of total width 2d). They are kept in order of y, which comes from the second sorted list, so no new sorting is needed.
6. **Examine the strip.** Each strip animal is compared with the animals that follow it in y-order, and the inner loop stops as soon as the vertical gap to the next animal is at least d. If any pair is closer than d, the best distance is updated.
7. **Return.** The smallest distance found in the halves or in the strip is the answer, together with the two animals that produced it.

To keep each level linear, the lists sorted by y are not sorted again at each level. Instead, the y-ordered list is split into a left and right list by a single pass that preserves its order (a stable partition). A left animal stays in the left list in the same relative order, so both lists are still sorted by y when the recursion needs them.

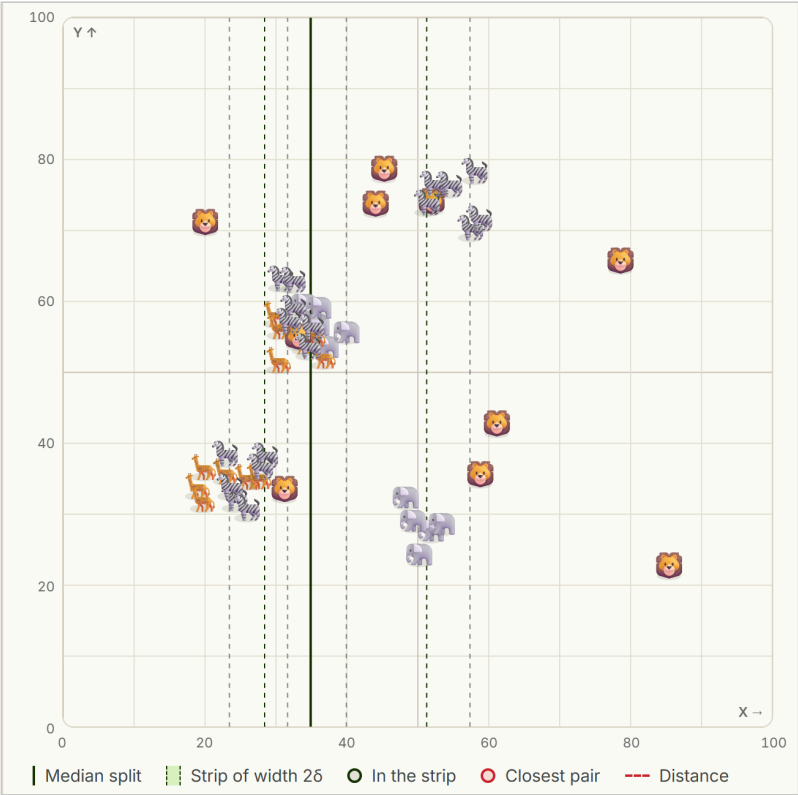


Figure 3. Closest-pair walkthrough at tick 101, step 3 (“Divide at the median x”). The solid green line is the median split at x = 34.886 (level 1); the dashed lines are the splits made inside each half (levels 2 and 3). Recursion stops when a piece holds three animals or fewer.

5.3 Why only a few strip animals need checking

The strip can contain many animals, so it is important that each one is compared with only a few others. Take a strip animal p and look only at animals above it whose y-difference from p is less than d. These animals lie in a rectangle of width 2d (d on each side of the dividing line) and height d. Divide this rectangle into eight squares with sides of d/2. Every square lies on one side of the dividing line, and any two animals on the same side are at least d apart, because d is the best distance found inside a half. The diagonal of a square is d/√2, which is less than d, so a square can hold at most one animal. Hence the rectangle holds at most eight animals, including p itself, and p needs to be compared with at most seven others. This is why the strip step costs only linear time.

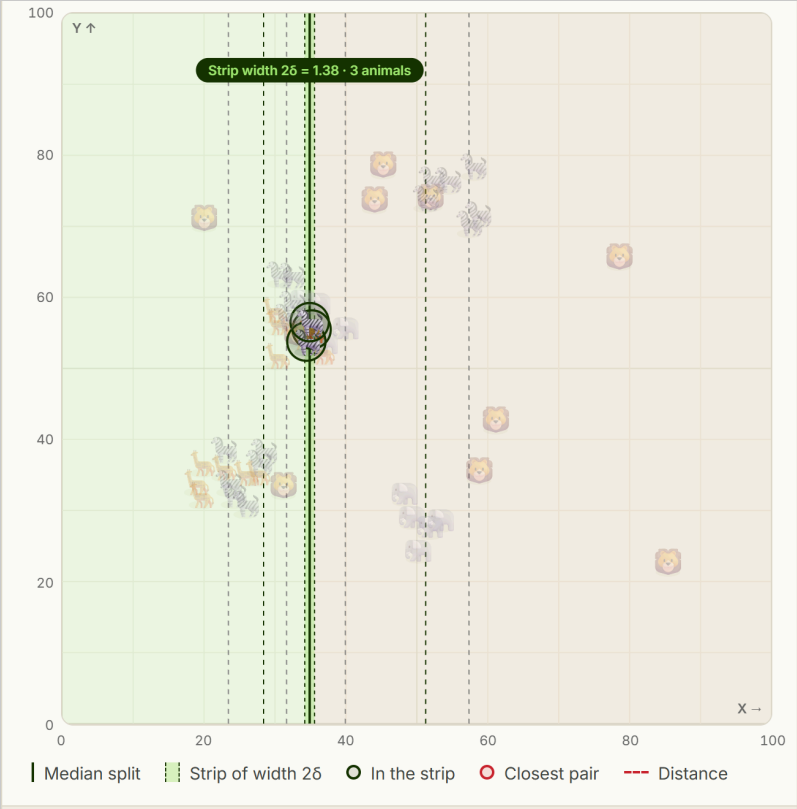


Figure 4. Closest-pair walkthrough at tick 101, step 6 (“Strip of width 2δ”). The shaded band around the median line is the strip; here it holds only three animals (ringed). Faded animals are outside the strip.

5.4 Complexity

Sorting by x and by y costs O(n log n) and is done once. The recursion halves the number of animals at each level, so there are about $\log_2 n$ levels. At every level, the partition of the y-list and the scan of the strip each take linear time in total, because each animal is in one strip at most and is compared with at most seven others. The running time therefore satisfies the recurrence

$$T(n) = 2\,T(n/2) + O(n)$$

whose solution is **O(n log n)**, including the initial sorting. This bound holds for every input and not only on average. The extra memory is O(n) for the sorted lists plus a recursion depth of about $\log_2 n$. Brute force, by comparison, is $O(n^2)$.

5.5 What the implementation records

Besides the answer, the code counts how much work it does. **Comparisons** counts every distance computation, both in the base cases and in the strip. **Recursive calls** counts the calls of the recursive function. These are observed values for one input and they are not a restatement of the theoretical bound. In addition, the top level of the recursion is traced so that the interface can explain it: the first three levels of dividing lines, the best pair found in the left and right halves, the value of d, the animals in the strip and which strip pairs were compared or skipped.

The Closest pair page turns this trace into an eight-step walkthrough: 1 original points, 2 sort by x and y, 3 divide at the median x, 4 left and right subproblems, 5 best distance d, 6 strip of width 2d, 7 compare inside the strip, and 8 the closest pair. Back, Next, Reset and Show result buttons (and the arrow keys) move between the steps. The result panels that the page shows for the sample state are included in Section 7 (Figure 8).

5.6 Pseudocode

Pseudocode: closest pair

```
CLOSEST-PAIR(P)
  Px ← P sorted by x ;  Py ← P sorted by y        // sorted once
  return SOLVE(Px, Py)

SOLVE(Px, Py)
  if |Px| ≤ 3: return the closest pair by brute force
  m ← x of the median point of Px                 // dividing line
  Lx, Rx ← left and right halves of Px ;  Ly, Ry ← Py split to match
  dL ← SOLVE(Lx, Ly) ;  dR ← SOLVE(Rx, Ry)
  d  ← min(dL, dR)
  S  ← points of Py with |x - m| < d               // the strip, in y-order
  for each p in S:
      for each later q in S while q.y - p.y < d:
          d ← min(d, dist(p, q))
  return d and the pair that gives it
```

6 KD-tree and rectangular range search

6.1 The problem and the idea of a spatial index

Given a rectangle with x between xmin and xmax and y between ymin and ymax, the problem is to report every point inside it, boundaries included. A linear scan tests all n points. That is acceptable for a single query, but if many rectangles are asked about, as when a ranger moves the zone, it is wasteful, because most points are nowhere near the rectangle. A **spatial index** is a data structure built once that lets a query ignore large parts of the plane. A KD-tree (k-dimensional tree) is such an index; here k = 2.

6.2 Construction

- The root splits the animals by x at the median animal: animals with a smaller x go to the left subtree and animals with a larger x to the right subtree.

- The two children split their own animals by y, the next level splits by x again, and so on. The splitting axis at depth d is d mod 2, so the axes alternate.
- The median animal at each step becomes the node itself, so every animal is stored in exactly one node. The 50 animals give a tree with 50 nodes. Using the median keeps the two sides about equal, so the tree stays balanced and, for 50 animals, has six levels.

Each node therefore owns a rectangular region of the plane, and its splitting line cuts that region in two (Figure 5). Region boundaries and split lines are exactly what the interface draws. To build the tree in O(n log n) the animals are sorted by x and by y once, as in the closest-pair algorithm. The median is then simply the middle index of the list for the current axis, and the other sorted list is split around that median with a stable linear partition, so nothing is sorted again. Each animal is also given a unique rank on each axis, which makes animals with equal coordinates split cleanly.

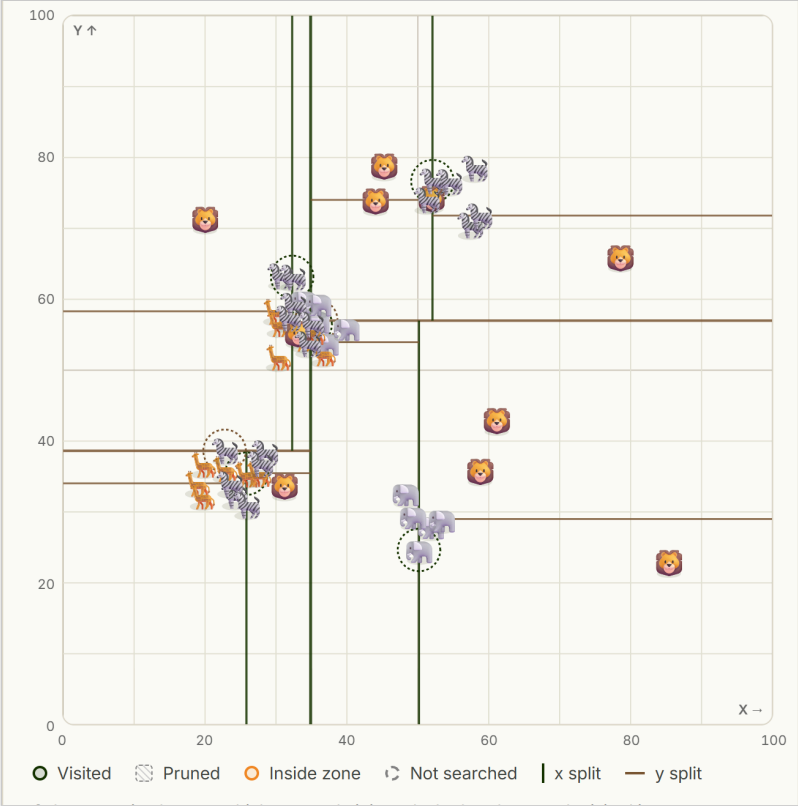


Figure 5. KD-tree walkthrough at tick 101, step 3 (“Alternating x and y axes”). Dark green lines are x splits and brown lines are y splits; each passes through the median animal of the region it splits. The legend below the plot explains the symbols used in the walkthrough.

6.3 The range query

The rectangle is described by its four bounds (the code swaps them if they are given in reverse order). The search starts at the root and works recursively:

1. **Visit the node.** The node is recorded as visited, and its own animal is tested against the rectangle, boundaries included. If it is inside, it is added to the result.
2. **Decide on the children.** Let s be the node’s splitting value on its axis, which is the x or y of the node’s animal. The left child holds animals on the low side of s, so it can contain matches only if the

rectangle's lower bound on that axis is at most s. The right child holds the high side, so it can contain matches only if the rectangle's upper bound is at least s.

3. **Prune or descend.** If the condition holds, the search continues in that child. Otherwise the whole subtree cannot touch the rectangle and is **pruned**: none of its animals is examined, and the root of the skipped subtree is recorded.
4. **Collect.** When the recursion ends, the animals gathered in step 1 are the answer.

Figure 6 shows the sample query. Animals with a dark ring were visited, hatched regions were pruned, and orange rings mark the nine animals inside the zone.

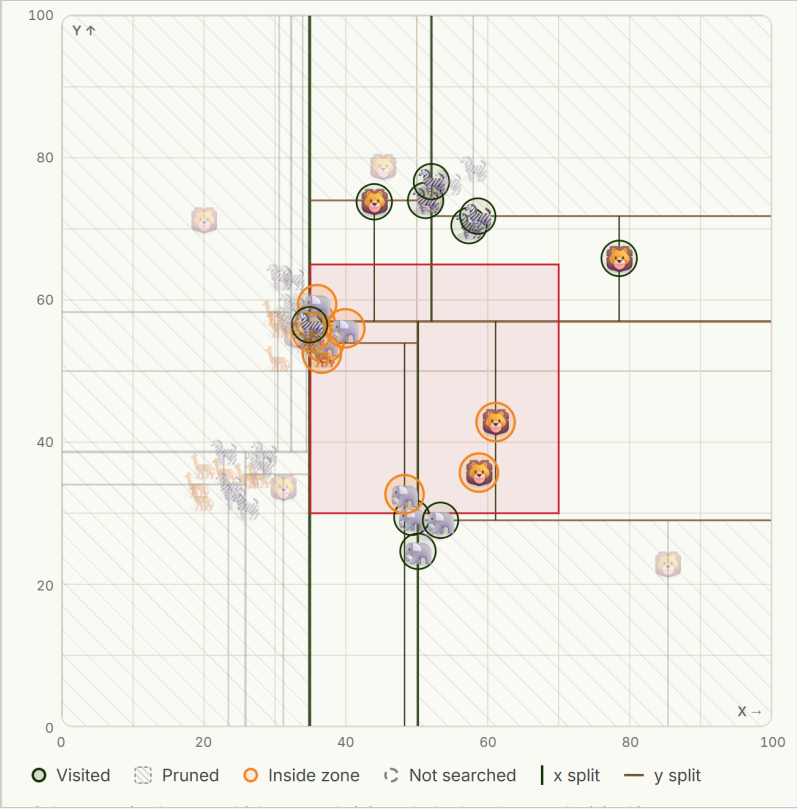


Figure 6. KD-tree walkthrough at tick 101, step 8 ("Matching points") for the zone x 35–70, y 30–65 (red rectangle). Ringed animals were visited, hatched regions were pruned, and orange rings mark the matches.

6.4 Complexity

Construction follows the same recurrence as the closest-pair algorithm, T(n) = 2T(n/2) + O(n), because every level does linear partitioning work, so it is **O(n log n)** and uses O(n) memory. For a balanced two-dimensional KD-tree the expected cost of a rectangle query is **O(√n + k)**, where k is the number of animals reported. Roughly, the edges of the rectangle cross about √n of the tree's regions and these must be examined, while regions completely inside or outside the rectangle are either all reported or skipped, and each reported animal costs one visit. This is the expected target for a balanced tree under suitable assumptions and not an unconditional guarantee: a rectangle that contains every animal has k = n, and the argument relies on the median splits keeping the tree balanced. A linear scan, in contrast, always tests all n animals. This report does not prove the bound; it measures operation counts, which are described in Section 7.

6.5 What the implementation exposes

The query returns the matching animals together with the ids of the visited nodes and of the roots of the pruned subtrees, plus counts of nodes visited, subtrees pruned and results. A separate call returns every node with its split axis, depth, region and subtree size, and the tree height. The KD-Tree search page uses this to run an eight-step walkthrough: 1 point set, 2 recursive median split, 3 alternating x and y axes, 4 the KD-tree, 5 range query, 6 visited nodes, 7 pruned subtrees, and 8 matching points. The rectangle can be redrawn on the plot, which runs a new query immediately. A “split lines” control (3 levels, 5 levels or all) limits how many levels of splitting lines are drawn so that the plot stays readable, and selecting an animal shows which node it is and whether the search visited it.

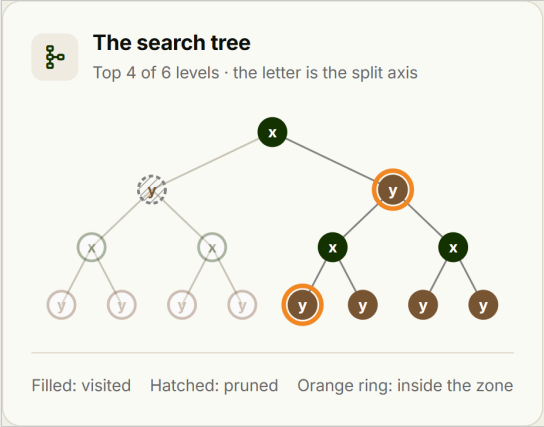


Figure 7. The top four levels of the KD-tree at tick 101 (the tree has six levels). The letter in each node is its split axis. Filled nodes were visited, the hatched node was pruned, and orange rings mark nodes whose animal lies inside the zone.

6.6 Pseudocode

Pseudocode: KD-tree construction and range search

```
BUILD(points, depth)
  if no points: return nil
  axis ← depth mod 2                              // 0 = x, 1 = y
  M ← median point of the list sorted on axis
  node ← (M, axis)
  node.left  ← BUILD(points before M, depth + 1)
  node.right ← BUILD(points after M, depth + 1)
  return node

RANGE-SEARCH(node, rectangle)
  if node.point is inside the rectangle: report it
  s ← node's split coordinate on its axis
  if rectangle.low[axis]  ≤ s: RANGE-SEARCH(node.left)   else: prune left subtree
  if rectangle.high[axis] ≥ s: RANGE-SEARCH(node.right)  else: prune right subtree
```

7 Sample input and output

This section lists a complete, reproducible sample. All values in it belong to one simulation state: seed 42, **tick 101 (t = 20.2 s)**, with the monitoring zone **x 35–70, y 30–65**, which is the application's default zone. The state can be reproduced by restarting the simulation, stepping to tick 101, and calling `GET /api/closest-pair?tick=101` and `POST /api/range-search` with the bounds above and tick 101.

7.1 Input

The input is the snapshot of the 50 animals at tick 101 (Table 3). Each animal is a point on the 100 × 100 plane. The animals tend to cluster in herds, and the lions are spread more widely.

Table 3. Input: positions of the 50 animals at tick 101. The first letter of the ID gives the species (E elephant, G giraffe, L lion, Z zebra).

ID	x	y	ID	x	y	ID	x	y	ID	x	y	ID	x	y
E01	50.16	24.64	G01	18.88	34.03	G11	29.89	58.30	L09	31.17	33.60	Z09	22.75	38.62
E02	51.82	27.98	G02	19.57	32.25	G12	30.39	56.63	L10	85.40	22.80	Z10	25.84	30.94
E03	49.29	29.43	G03	19.73	36.75	L01	58.75	35.71	Z01	51.24	74.01	Z11	28.40	38.39
E04	48.27	32.68	G04	25.83	35.40	L02	44.00	73.82	Z02	52.06	76.65	Z12	27.70	36.80
E05	53.34	29.00	G05	22.77	36.13	L03	78.53	65.86	Z03	57.36	70.47	Z13	34.42	53.77
E06	35.69	56.99	G06	27.52	35.47	L04	51.91	74.19	Z04	58.59	71.81	Z14	32.40	59.13
E07	36.97	53.95	G07	33.91	54.50	L05	20.00	71.25	Z05	54.37	76.57	Z15	30.59	63.36
E08	33.91	59.87	G08	36.61	52.45	L06	45.22	78.73	Z06	57.95	78.50	Z16	32.28	63.16
E09	39.92	56.02	G09	35.16	55.48	L07	61.10	42.82	Z07	24.09	32.31	Z17	34.89	56.51
E10	35.95	59.41	G10	30.28	51.84	L08	33.06	55.14	Z08	23.43	34.01	Z18	31.62	57.33

The query for the range search is the rectangle with xmin = 35, xmax = 70, ymin = 30 and ymax = 65.

7.2 Output of the closest-pair algorithm

Table 4 lists what the closest-pair code returned for the snapshot. Both animals of the closest pair are in the right half, so the answer was already found by the recursion and the strip step only confirmed it. In other states the closest pair crosses the dividing line and is found in the strip.

Table 4. Closest-pair output at tick 101.

Quantity	Value
Closest pair	Z01 (zebra) and L04 (lion)
Distance	0.691 units
Distance computations (comparisons)	57, against 1,225 pairs for brute force
Recursive calls	35
Median split at the top level	x = 34.886
Best pair in the left half	G07 and Z13, distance 0.892
Best pair in the right half	Z01 and L04, distance 0.691
Strip around the median (width 2δ = 1.38)	3 animals; no pair needed comparing, so the strip did not improve the answer

7.3 Output of the range search

Table 5 lists the result of the range query. The nodes visited include the nine matching animals and ten others that were visited only to decide where to search next. The closest pair and the range result were each compared with a simple method (brute force over all pairs, and a linear scan of all animals) and the results were identical.

Table 5. Range-search output at tick 101.

Quantity	Value
Query rectangle	x 35 to 70, y 30 to 65
Animals inside (k)	9 of 50: E04, E06, E07, E09, E10 (elephants), G08, G09 (giraffes), L01, L07 (lions)
Nodes visited	19, against 50 tests for a linear scan
Subtrees pruned	4
Tree	50 nodes, 6 levels, root Z17

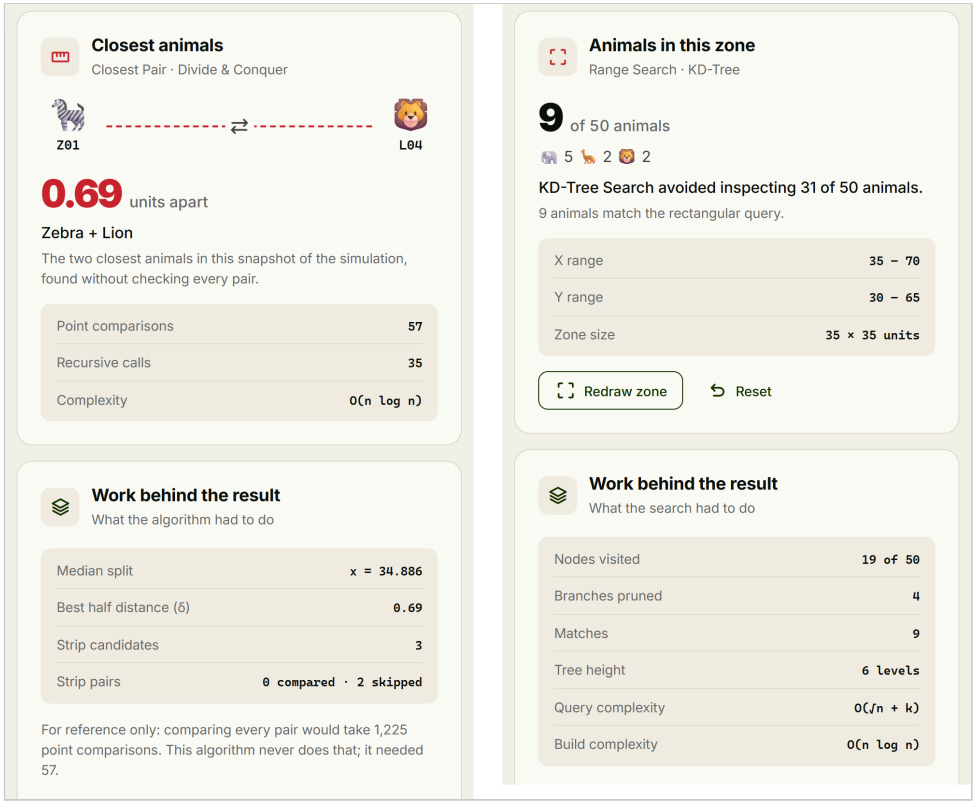


Figure 8. How the application shows the two outputs at tick 101. Left: the Closest pair page (the closest animals and the work behind the result). Right: the KD-Tree search page (animals in the zone and the work behind the result). The values are those in Tables 4 and 5.

7.4 Observations

Theory versus observation. The bounds O(n log n) and O(√n + k) are statements about how the work grows with n. The numbers in Tables 4 and 5 are counts for one input of 50 animals: 57 distance computations instead of 1,225, and 19 node visits instead of 50. They show the saving but are not proof of a bound, and with only 50 animals the absolute savings are small.

The result changes as the animals move. The same code was run at other ticks of the same seed, with the same zone (Table 6). The closest pair, the number of animals in the zone and the work done all change with the positions, while the number of recursive calls is 35 every time, because the shape of the recursion depends only on the number of animals.

Table 6. The same analyses at five ticks of the seed-42 run (each row is a separate simulation state). Every result matched brute force and a linear scan.

Tick	Closest pair	Distance	Comparisons	Animals in zone	Nodes visited
1	G07, Z11	0.593	64	18	37
101	Z01, L04	0.691	57	9	19
251	Z16, L04	0.789	60	12	32
501	Z13, Z18	1.710	68	4	17
1001	G03, G02	1.759	65	8	24

Verification. The script `backend/verify_algorithms.py` checks the implementations against simple methods. It tests the closest-pair code on 600 random cases, including duplicate points and points with equal coordinates, and the KD-tree on 3,000 random range queries, whose bounds are taken from actual point coordinates to test the inclusive boundaries. It also tests snapshots of the simulation at eight ticks, and every result matched brute force or a linear scan.

Growth on larger inputs. The same script also runs both algorithms on random points in a square for larger n and prints operation counts (Table 7). The range query is a box covering 5% of each side, so about 0.25% of the points match, averaged over 20 boxes. These are counts of distance computations and node visits and not running times.

Table 7. Observed operation counts on random points (output of verify_algorithms.py).

n	Closest-pair comparisons	Tree height	Nodes visited	√n
1,000	1,134	10	20	32
4,000	4,617	12	35	63
16,000	18,813	14	82	126
64,000	76,105	16	238	253

The closest-pair comparison count stays at about 1.1 to 1.2 per point; it counts only distance computations, not the sorting steps, so it is lower than n log n. The tree height equals $\log_2 n$ rounded up, which shows that the tree is balanced. The number of nodes visited by the small query stays below $\sqrt{n}$ plus the number of matches. These counts are consistent with the stated complexities but they are only one set of random inputs.

8 Discussion

The two algorithms answer different questions about the same set of points. The closest pair is one global question with a single answer. Divide and conquer solves it by cutting the problem in half, solving the halves, and noticing that only the thin strip around the cut can hide a better answer. The range search is a question that can be repeated for any rectangle. Its cost is paid in building the tree once, after which each query looks only at the parts of the plane that the rectangle can touch. Both rely on the same idea: the sorted order of the points lets large amounts of work be skipped.

The two operation counts illustrate this. For the sample state the closest pair needed 57 distance computations instead of 1,225, about 5% of the brute-force work, and the zone query visited 19 nodes instead of testing 50 animals. They also show why the theoretical bound and the observed count are different things: the bound describes growth, while the count depends on where the animals happen to be, which is why it varied from 57 to 68 across the five sampled ticks.

Because the animals move, the KD-tree is rebuilt for every snapshot. It is therefore a query structure for one moment and not an index updated as the animals move. With 50 animals a linear scan would also be fast, so the project is a demonstration of the methods and not a speed benchmark.

The interface is meant to make the algorithms understandable. For closest pair it shows the median line, the subproblems, the value of d, the strip and the final pair on the same animals that the numbers describe. For the KD-tree it shows the splitting lines, the query rectangle, the visited nodes and the pruned regions. The Live habitat page shows how the answers change as the animals move.

9 Limitations

- **Simulated data.** The animals, herds and movement come from a seeded model. There is no real tracking data, and the results say nothing about real animals.
- **Simplified geometry.** The habitat is a flat 100 × 100 plane with abstract units and no terrain, obstacles or geography. The movement model contains only herding, personal space and steering away from the edges; it has no predator–prey behaviour, feeding or time of day.
- **Fixed small size.** The population is always 50 animals, so the interface cannot show how the algorithms scale. Scaling was examined only through operation counts on random inputs (Table 7), and no running times were measured.
- **Static analysis of each snapshot.** The KD-tree is rebuilt for every snapshot and is not updated incrementally. The O(√n + k) bound is argued, not proved, in this report.
- **Limited testing.** Correctness was checked with `backend/verify_algorithms.py`; there is no larger automated test suite and no user study.
- **Local use only.** The simulation runs in one backend process with the last 3,000 ticks in memory. The application has only been run locally.

10 Conclusion

GeoWild implements two spatial algorithms from scratch in Python and demonstrates them on a live, reproducible simulation. The closest pair of points is found by divide and conquer in O(n log n) time by sorting once, splitting at the median, solving both halves, and checking only the strip around the dividing line, where each animal needs at most seven comparisons. The two-dimensional KD-tree is built in O(n log n) time and answers rectangular range queries by pruning regions that cannot contain matches, with an expected cost of O(√n + k) on a balanced tree.

Both algorithms were checked against simple reference methods, and their operation counts were reported next to the theory. The step-by-step views of the interface are generated from traces produced by the algorithms themselves, so a reader can follow the divide, the strip, the splits and the pruning on the same data as the reported numbers. As a teaching tool the project connects each algorithm's code with a picture of what it does.

References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*.
- M. de Berg et al., *Computational Geometry: Algorithms and Applications*.
- J. L. Bentley, "Multidimensional binary search trees used for associative searching", *Communications of the ACM* 18(9), 1975.